Lecture 17: Ensemble Learning and Random Forests

Md. Shahriar Hussain
ECE Department, NSU

What is Ensemble Learning

- Scenario: you need to buy a smartphone. Do you…

  a) walk into the nearest store and buy the first smartphone the salesperson shows you?
  b) Extensively research the latest models, check reviews and specs online, consult friends and family, and THEN walk over to buy the phone?

- If you do (b), you're making decisions by an ensemble (i.e. "wisdom of the crowd") – instead of a single factor (i.e. the salesperson)

- Aggregating predictions of a group of predictors (ensemble) are better overall than the best individual predictor

- Ensemble models in machine learning combine the decisions from multiple models to improve the overall performance.

Why use Ensemble Learning?

- Law of large numbers reduces probability for making a wrong prediction
- It combines few good predictors which eventually provides a better result
- Ensemble methods work best when the predictors are as independent from one another as possible.
- One way to get diverse classifiers is to train them using very different algorithms.
- This increases the chance that they will make very different types of errors, improving the ensemble's accuracy.

Classifier Independence

- Classifiers are not guaranteed to be independent of each other – i.e. errors may be correlated

- However, we can:
  1. Use different algorithms → every algorithm has its own strengths and weaknesses, having a variety will help achieve lower error rate
  2. Use different features → different features capture different aspects of a particular class – using a variety helps!
  3. Use different training data → subject different models to different subsets of training data to reduce variance error

Recall: Bias/Variance Trade-off

ML model's generalization error comes from three sources:

1. Bias → generalization error due to wrong assumptions
   - e.g. assuming the data is linear when it's actually quadratic
   - High bias will lead to underfitting
2. Variance → high sensitivity to small variations in training data
   - e.g. polynomial model with many degrees of freedom on quadratic data
   - High variance leads to overfitting
3. Irreducible error → due to inherent noise of the data itself

- Make models more complex → higher variance, lower bias, overfitting
- Make models less complex → lower variance, higher bias, underfitting
- Need to strike a very delicate balance between the two

How to predict with an Ensemble?

- Majority voting
  - Return the most popular prediction from multiple prediction algorithms
- Bootstrap Aggregation – aka Bagging
  - Resample data to train algorithm on different random subsets
- Boosting
  - Reweight data to train algorithm to specialize on “hard” examples – predecessor mistakes
- Stacking
  - Train a model that learns how to aggregate classifiers’ predictions

Majority Voting

- Hard voting classifier: aggregate predictions of each classifier, and predict the class that gets the most votes
  - Often achieves higher accuracy than the best classifier in the ensemble
  - Even if each classifier is a weak learner, the ensember can be a strong learner
  - If trained on the same data, however, their errors are likely to be correlated!

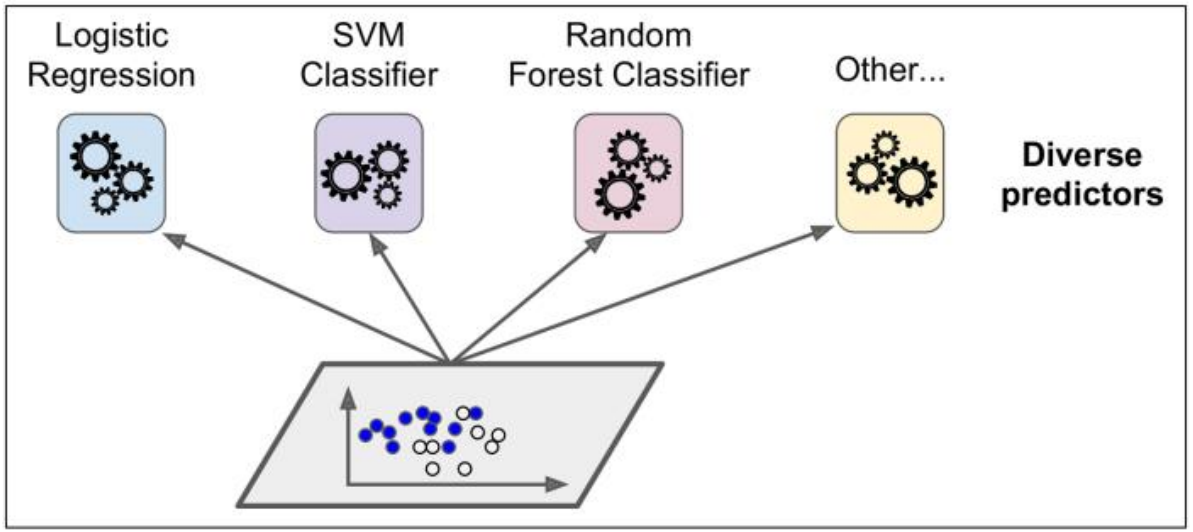


Figure 7-1. Training diverse classifiers

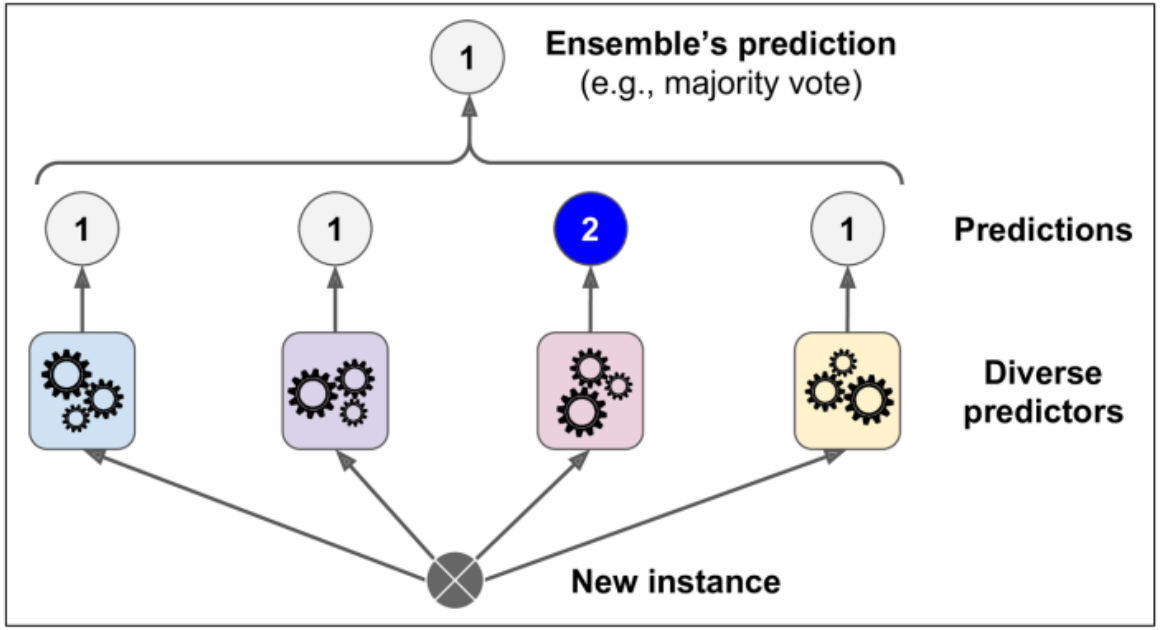


Figure 7-2. Hard voting classifier predictions

Example

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

log_clf = LogisticRegression()
rnd_clf = RandomForestClassifier()
svm_clf = SVC()

voting_clf = VotingClassifier(
   estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
   voting='hard')

voting_clf.fit(X_train, y_train)
```

Example

- Each classifier's accuracy on the test set

```
>>> from sklearn.metrics import accuracy_score
>>> for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
... clf.fit(X_train, y_train)
... y_pred = clf.predict(X_test)
... print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
...
```

- LogisticRegression 0.864
- RandomForestClassifier 0.896
- SVC 0.888
- VotingClassifier 0.904

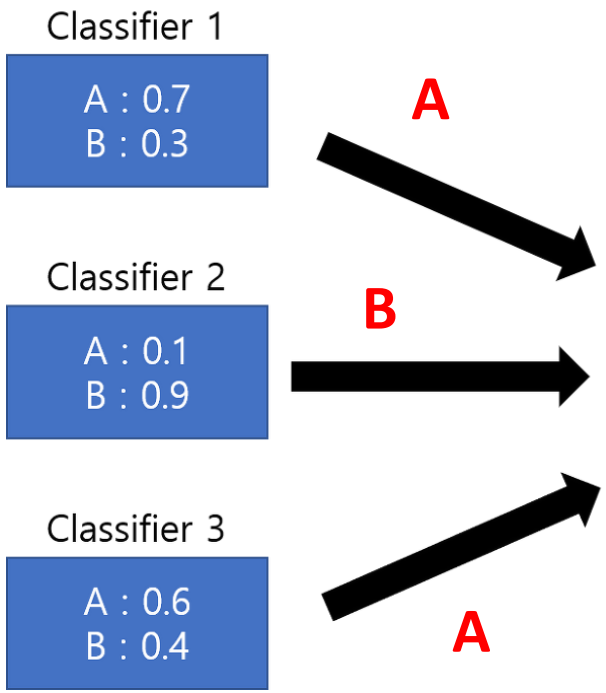
Majority Voting

- Soft voting classifier:
  - predict the class with the highest class probability, they all have a *predict_proba()* method
  - averaged over all individual classifiers

- It often achieves higher performance than hard voting because it gives more weight to highly confident votes

- replace *voting="hard"* with *voting="soft"*

Hard Voting vs Soft Voting



$$A = \frac{0.7+0.1+0.6}{3} = 0.47$$

$$B = \frac{0.3+0.9+0.4}{3} = 0.53$$

Predicted Output:

- Hard voting A
- Soft voting B

Bagging

- Short for Bootstrap AGGregatING
- One way to get a diverse set of classifiers is to use very different training algorithms. Another approach is to use **the same training algorithm** for every predictor and train them on **different random subsets** of the training set.
- For Bagging Use same algorithm on different random subsets of training set
- When sampling is done with replacement, it's bagging
- When sampling is done without replacement, it's pasting
- Prediction made like majority voting

Bagging and Pasting

- Once all predictors are trained, the ensemble can make a prediction for a new instance by simply aggregating the predictions of all predictors.

- Predictors can all be trained in parallel, via different CPU cores or even different servers. Similarly, predictions can be made in parallel.

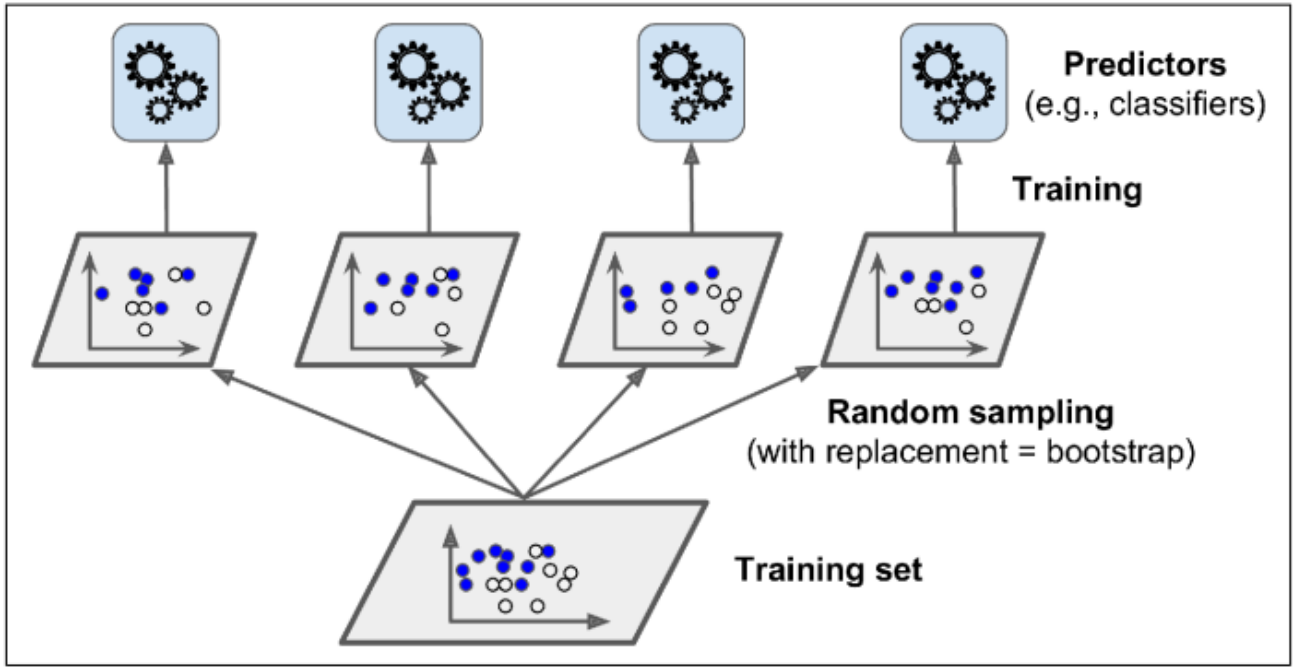


Figure 7-4. Bagging and pasting involves training several predictors on different random samples of the training set

Bagging and Pasting in Scikit-Learn

- Scikit-Learn offers a simple API for both bagging and pasting with the BaggingClassifier class

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
bag_clf = BaggingClassifier(
  DecisionTreeClassifier(), n_estimators=500,
  max_samples=100, bootstrap=True, n_jobs=-1)
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

Hyperparameter: Bagging and Pasting

- The code trains an ensemble of 500 Decision Tree classifiers
- Each is trained on 100 training instances randomly sampled from the training set with replacement (For bagging)
- For pasting, set bootstrap=False
- The n_jobs parameter tells Scikit-Learn the number of CPU cores to use for training and predictions (–1 tells Scikit-Learn to use all available cores)

Bagging Example

- Build ensemble from “bootstrap samples” drawn with replacement
- Duplicate data can occur for training for bagging
- No duplicate data in pasting
- Some indices may be missing from training data
- Each classifier is trained on a different subset of data

Sample indices	Bagging Round 1	Bagging Round 2	…	Bagging Round n
1	2	7	…	4
2	2	3	…	3
3	1	2	…	1
4	3	1	…	1
5	7	1	…	2
6	2	7	…	7
7	4	7	…	6

Out-of-Bag Evaluation

- With bagging, some instances may be sampled several times for any given predictor, while others may not be sampled at all.
- the training instances that are not sampled are called *out-of-bag (oob) instances.*
- Since a predictor never sees the oob instances during training, it can be evaluated on these instances, without the need for a separate validation set.
- In Scikit-Learn, we can set *oob_score=True* when creating a BaggingClassifier to request an automatic oob evaluation after training

Random Patches and Random Subspaces

- The BaggingClassifier class supports sampling the features as well.
- Sampling is controlled by two hyperparameters: *max_features* and *bootstrap_features*
- They work the same way as max_samples and bootstrap, but for feature sampling instead of instance sampling.
- Thus, each predictor will be trained on a random subset of the input features
- Sampling both training instances and features is called the ***Random Patches method***
- Keeping all training instances but sampling features is called the ***Random Subspaces* method**

Random Forests

- As an example of an Ensemble method, we can train a group of Decision Tree classifiers, each on a different random subset of the training set.
- To make predictions, we obtain the predictions of all the individual trees, then predict the class that gets the most votes.
- Such an ensemble of Decision Trees is called a ***Random Forest***

- A **Random Forest** is an ensemble of **Decision Trees**,
  - generally trained via the **bagging method** (or sometimes pasting),
  - typically with ***max_samples*** set to the **size of the training set**

Random Forests

- Instead of building a BaggingClassifier and passing it a DecisionTreeClassifier, you can instead use the RandomForestClassifier class

```
from sklearn.ensemble import RandomForestClassifier
rnd_clf = RandomForestClassifier(n_estimators=500, max_leaf_nodes=16,
    n_jobs=-1)
rnd_clf.fit(X_train, y_train)

y_pred_rf = rnd_clf.predict(X_test)
```

Random Forests

Original Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Bootstrapped Dataset

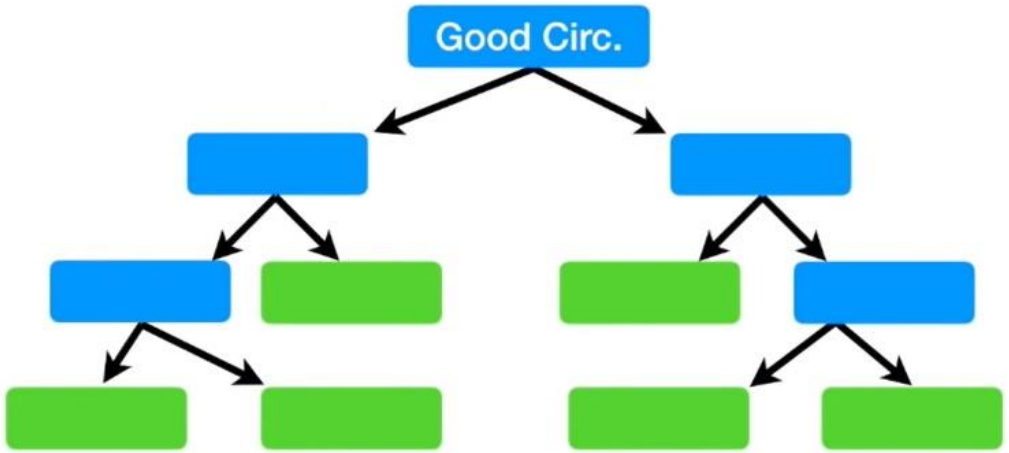
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

Random Forests

We built a tree…

1) Using a bootstrapped dataset

2) Only considering a random a subset of variables at each step.



Bootstrapped Dataset

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
Yes	Yes	Yes	180	Yes
No	No	No	125	No
Yes	No	Yes	167	Yes
Yes	No	Yes	167	Yes

Random Forests